

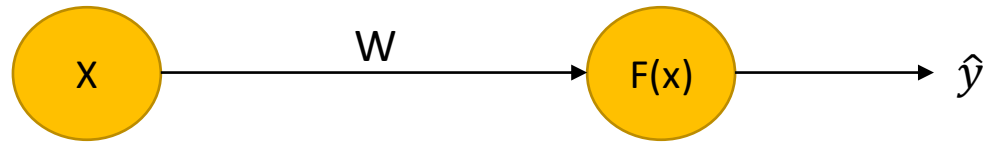
Deep Learning and Neural Networks

Priyanka Bagade, IITK

Lecture 5

Errors and Weights

- Perceptron with a single input



- Suppose input $x = 0.3$, true output, $y = 0.8$, predicated output for the given perceptron can be calculated using feedforward process:
 - $y_i = w.x = w.0.3$
 - Corresponding error

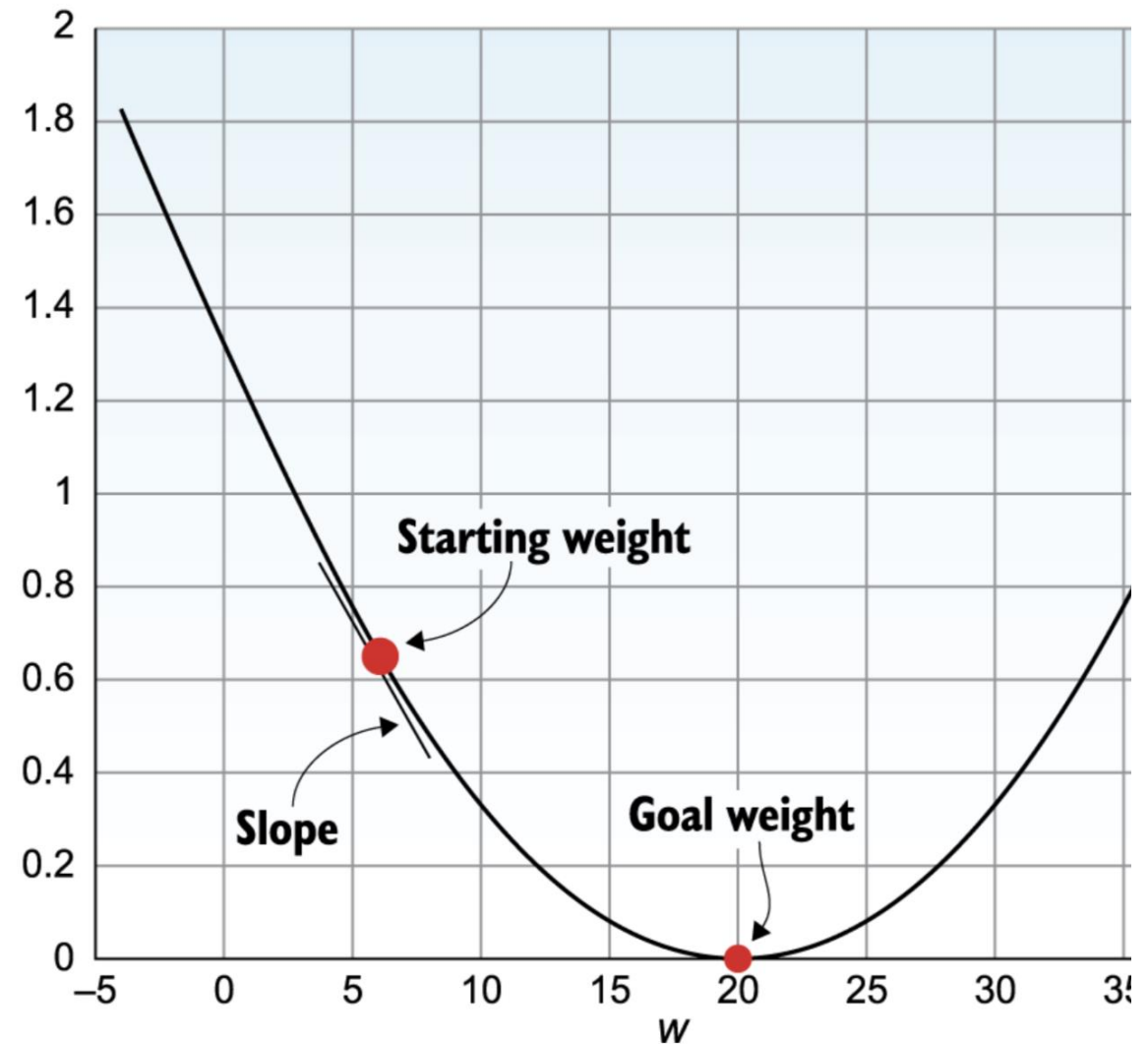
$$\begin{aligned} error &= |\hat{y} - y| \\ &= |(w.x) - y| \\ &= |w.0.3 - 0.8| \end{aligned}$$

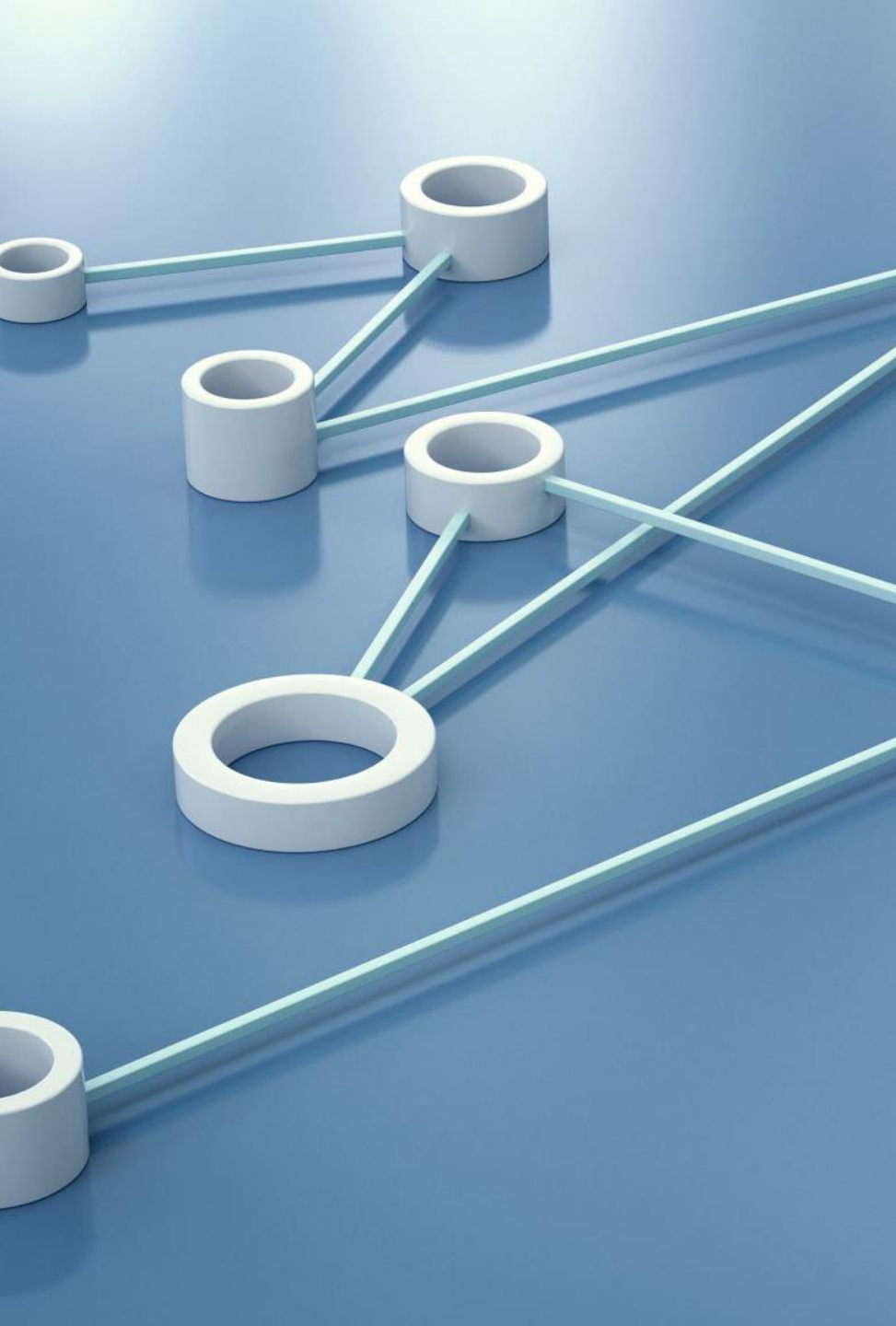
Input x and expected output y
and constants

Network learns by adjusting the
weights

Cost/Error function vs weight plot

- Initialize the network with random weights
- Aim to descend the curve and find the optimal value of w to get the minimum error
- The process of iterating through different weight values to reach the minimum error is called as *optimization algorithm*

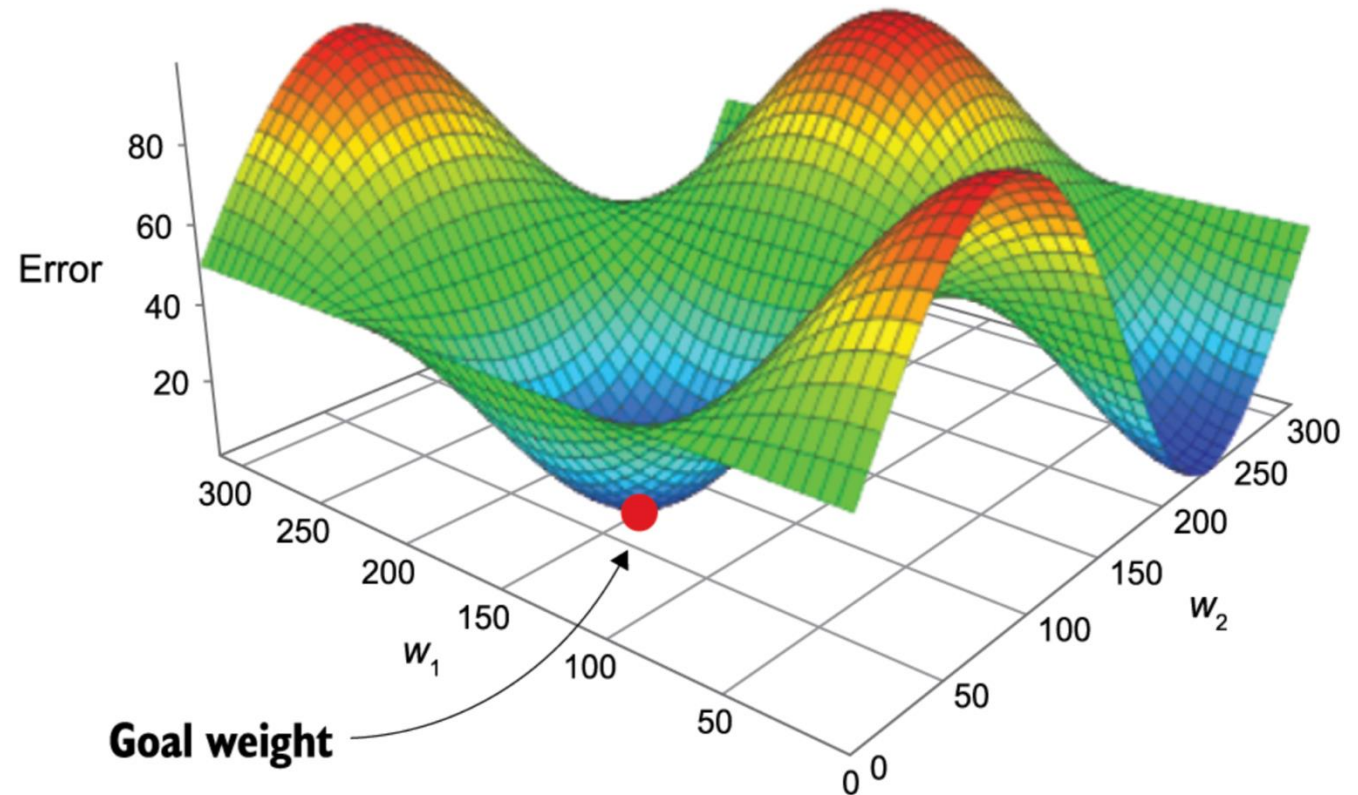
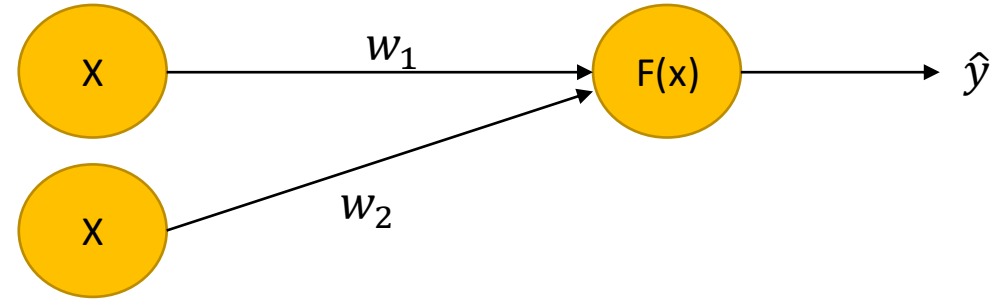
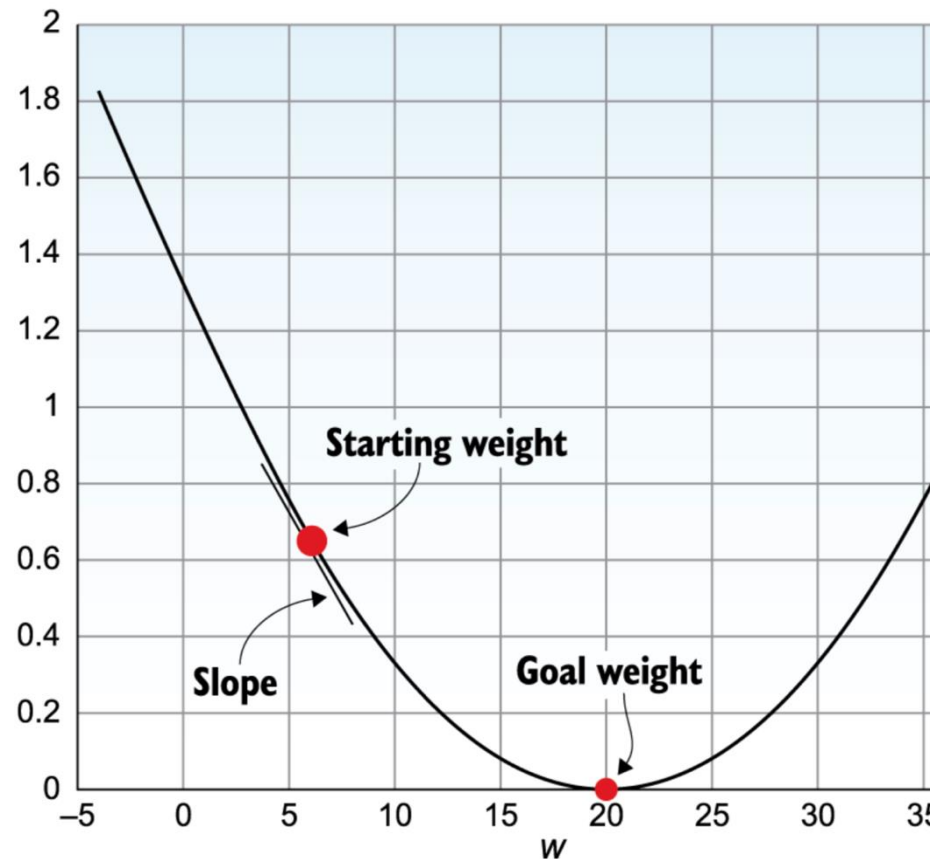
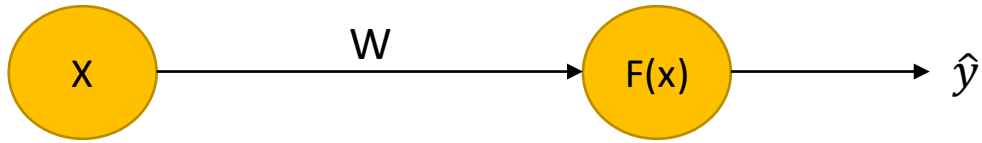




Optimization Algorithms

- Training neural network
 - Process multiple data samples
 - Make predictions using feedforward process
 - Calculate error
 - Adjust weights to minimize the error and improve the prediction accuracy
- Optimization
 - Way of formalizing a problem to minimize or maximize the value
 - First define the metric that needs to be optimized (e.g. cost, time or error, distance)
 - Based on the error function, adjust the parameters

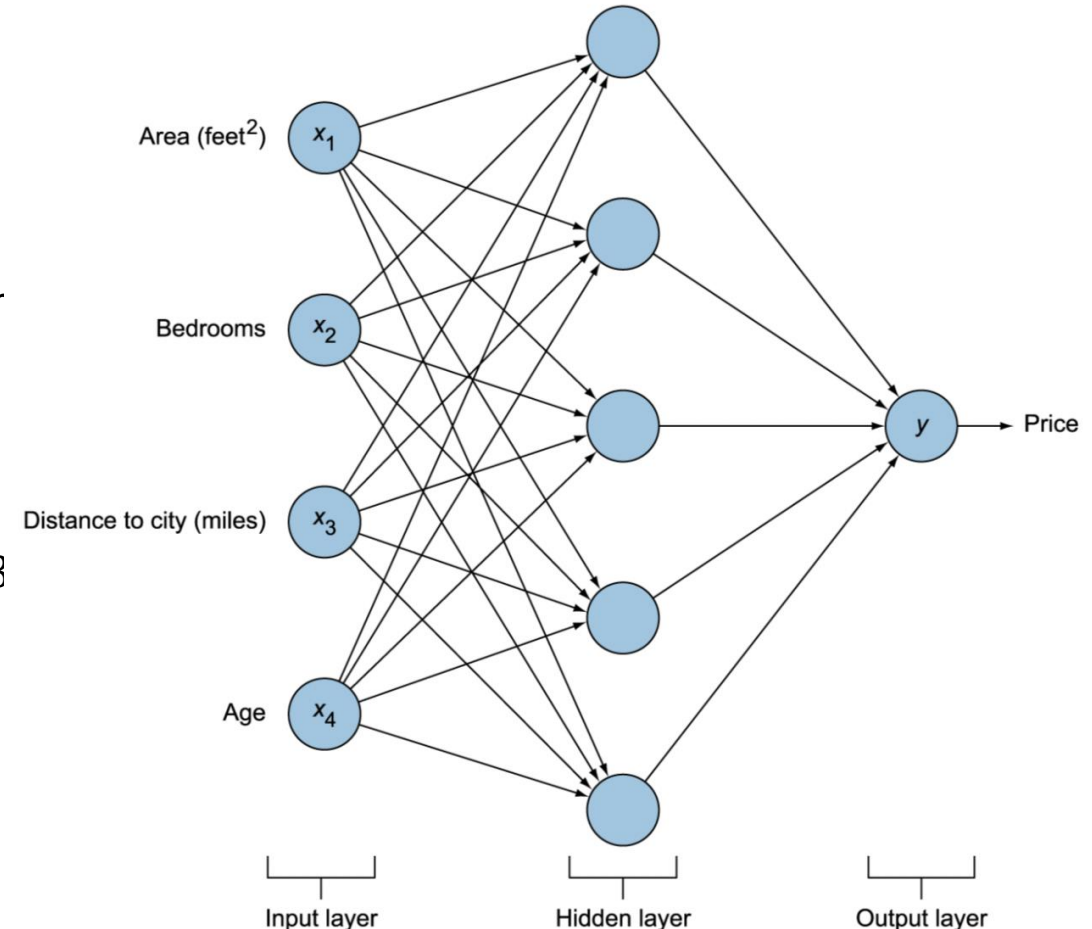
Optimization for Neural Networks



Why do we need optimization algorithm?

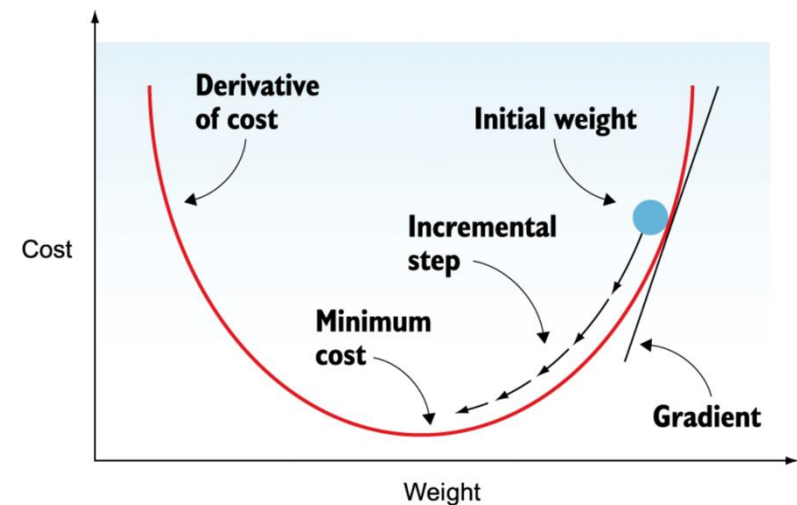
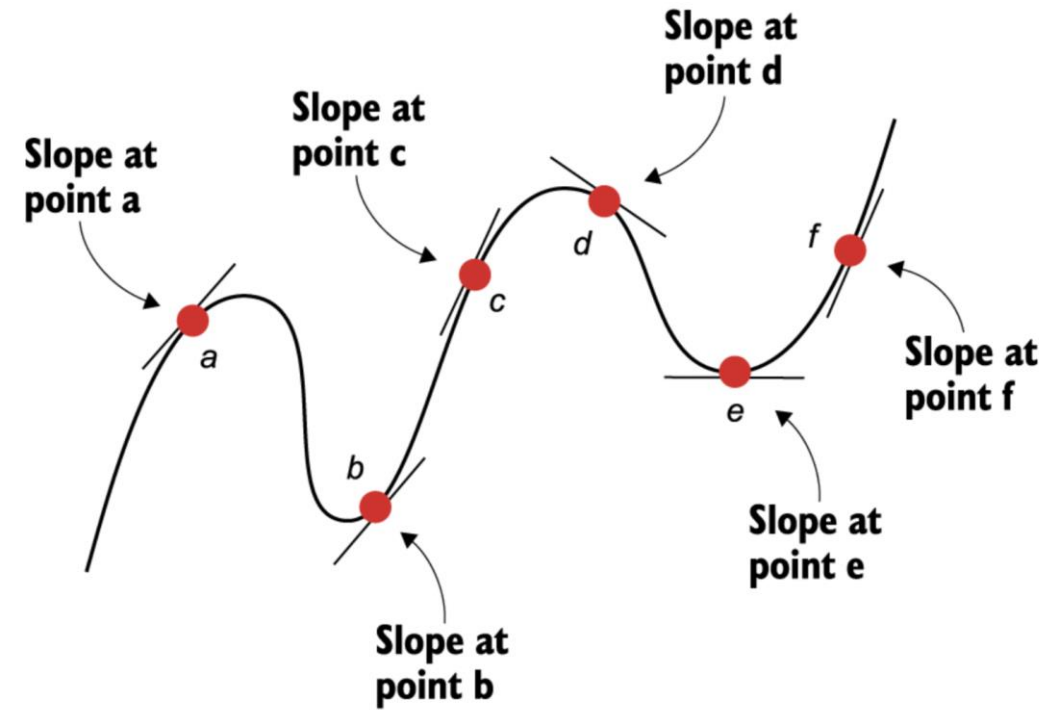
- Can we use brute force method?
 - Yes, when we have very few inputs and 1 or 2 neurons
- House pricing prediction example
 - 20 edges from input layer to hidden layer + 5 edge from hidden layer to output layer = 25 edges = 25 weights
 - If we try 1000 values for each weight parameter, total combinations would be,
 $1000 \times 1000 \times \dots \times 1000 = 1000^{25}$
 - For the super fast computer with the speed of 93 petaflops (93 10^{15} floating point operations per second), it will take

$$\frac{10^{75}}{93 \times 10^{15}} = 1.08 \times 10^{58} \text{ seconds} = 3.42 \times 10^{50} \text{ years}$$



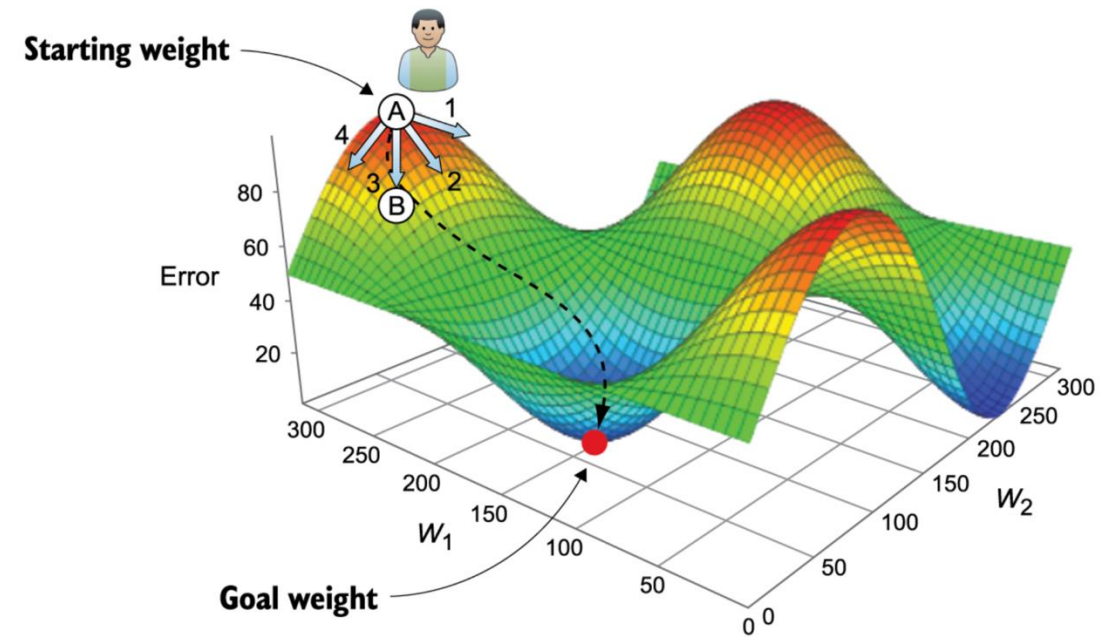
Batch gradient descent

- Gradient/derivative slope of the curve at any given point
- Gradient descent helps update the weights iteratively to descend the slope of the error curve to obtain minimum error in the prediction
- At the initial weight point calculate the derivative of the error function to get the slope i.e. direction of the next step.
- Calculate the output for the new weights
- Again, calculate the next step (slope) and repeat the process until we get the minimum error



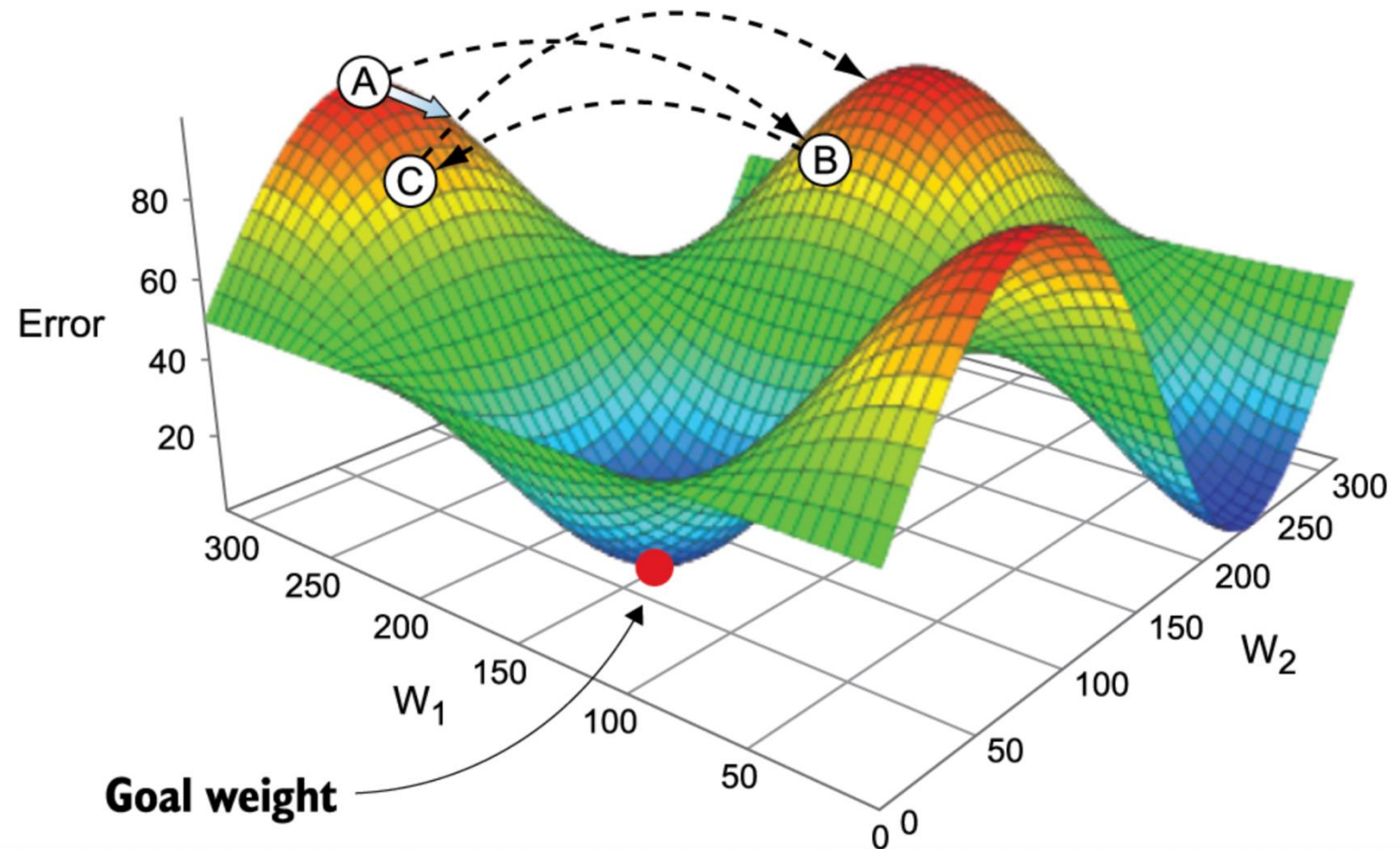
How does gradient descent work?

- Start with random weight values for w_1 and w_2 at point A.
- Descend the error mountain until minimum error is achieved (take multiple steps down to reach the minimum error point)
- Gradient (step direction)
 - $\frac{dE}{dw}$
- Learning rate (step size) - α
 - Size of the step the networks takes when descending the error mountain
 - An important hyperparameter to learn
 - Large step – faster learning
 - Small step – slower learning



Learning rate

- Large learning rate
 - Oscillating error
 - Will never descent and never get trained
- Small learning rate
 - Eventually descent the error mountain and obtain minimum error
 - Disadvantage- longer training time
- Putting direction and step together. Change in weight for every step is given by,
$$\Delta w_i = -\alpha \frac{dE}{dw_i}$$
 - Negative sign is added as slope is always calculated in the upward direction

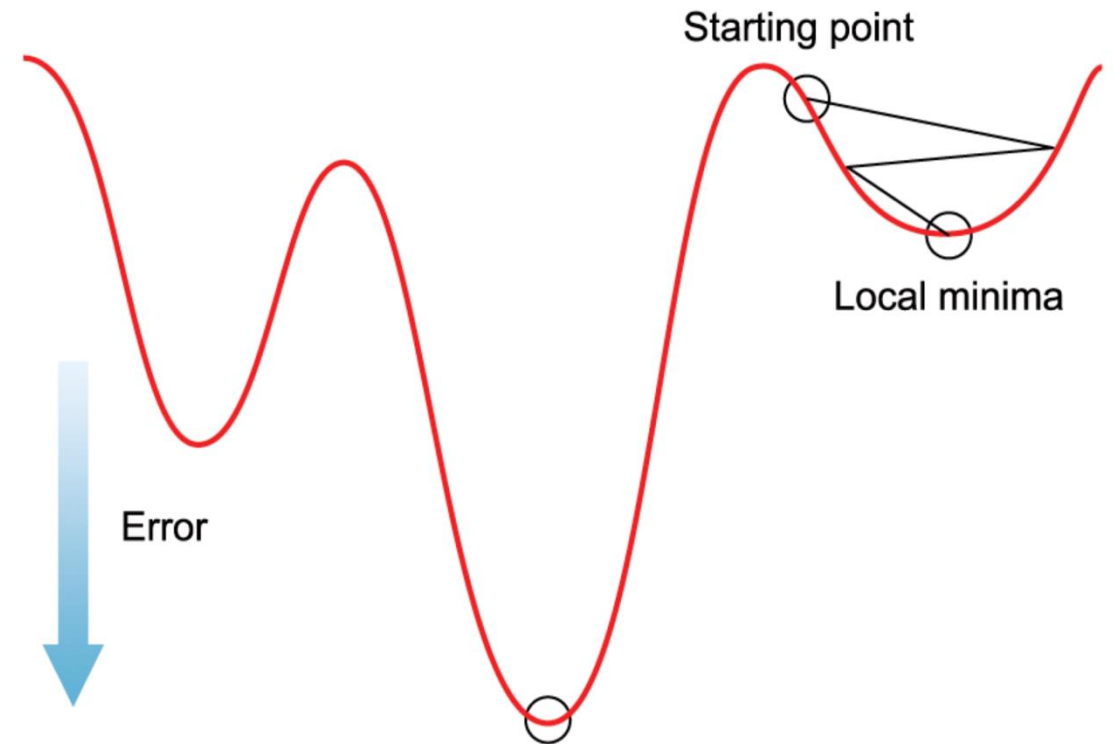


Pitfalls of batch gradient descent (BGD)

- For the irregular terrain in the cost function plot, BGD can end up finding local minima
- BGD uses entire training set to compute gradient at every step
- Loss function

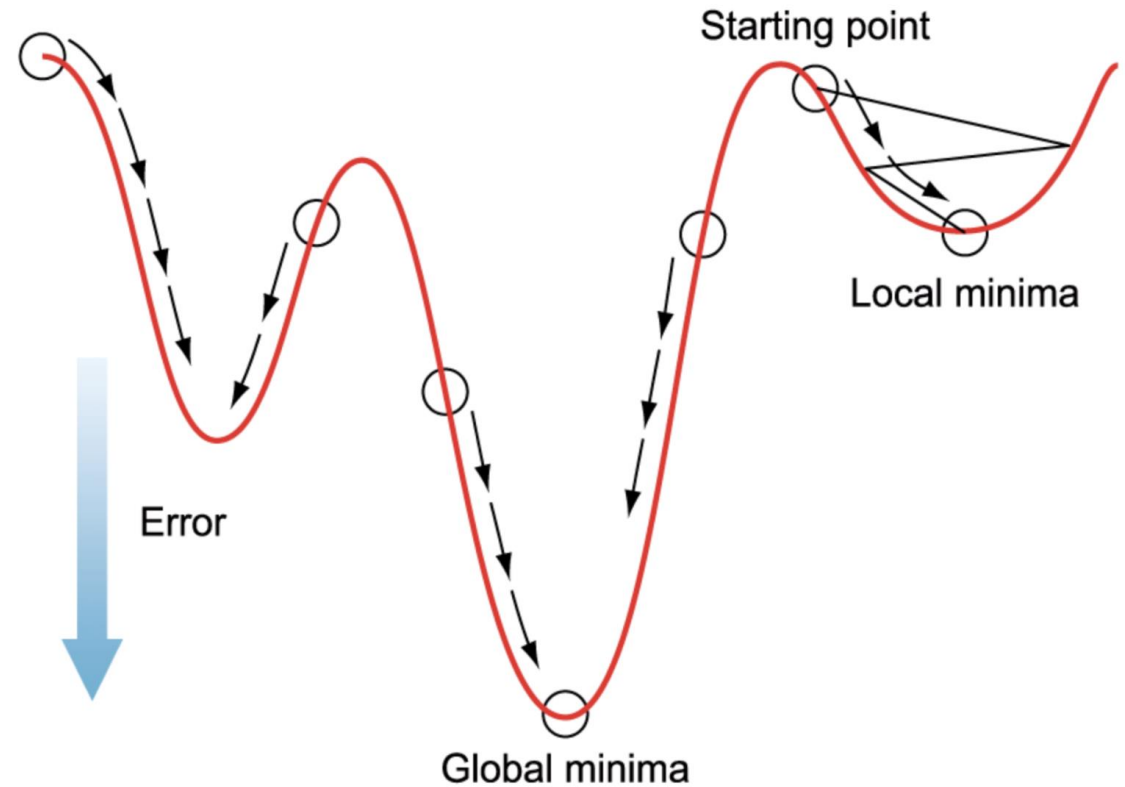
$$E(W, b) = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2$$

- Slow and computationally heavy – e.g. for 100 million training samples, the algorithm will sum error for 100 million samples just to take a single step
- Processes the entire training data in a single batch thus called as batch gradient descent



Stochastic gradient descent (SGD)

- Stochastic == Random
- Randomly selects data points and runs the gradient descent algorithms for a single data point at a time
- Multiple weight starting points that lead to multiple local minimas
- Minimum value of all the local minimas gives the global minima
- SGD almost always performs better than BGD



GD	Stochastic GD
1. Take all the data. 2. Compute the gradient. 3. Update the weights and take a step down. 4. Repeat for n number of epochs (iterations). A smooth path for the GD down the error curve	1. Randomly shuffle samples in the training set. 2. Pick one data instance. 3. Compute the gradient. 4. Update the weights and take a step down. 5. Pick another one data instance. 6. Repeat for n number of epochs (training iterations). An oscillated path for SGD down the error curve

Mini-batch gradient descent (MB-GD)

- Compromise between BGD and SGD
- Divide the training dataset into small batches and compute the gradient
- Converges in fewer iterations as compared to BGD
- Computation performance again over SGD

Backpropagation

- Feedforward

$$\hat{y} = \sigma \cdot W^{(3)} \cdot \sigma \cdot W^{(2)} \cdot \sigma \cdot W^{(1)} \cdot (x)$$

- Predict the error – loss function

$$E(W, b) = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2$$

- Gradient descent algorithm to compute optimal w to minimize error

$$\Delta w_i = -\alpha \frac{dE}{dw_i}$$

- Update all the weights in the network using backpropagation

The diagram shows the weight update formula: $W_{new} = W_{old} - \left(\frac{\partial Error}{\partial W_x} \right)$. Arrows point from labels to parts of the formula: 'Old weight' points to W_{old} , 'Derivative of error with respect to weight' points to $\left(\frac{\partial Error}{\partial W_x} \right)$, 'New weight' points to W_{new} , and 'Learning rate' points to the minus sign.

$$W_{new} = W_{old} - \left(\frac{\partial Error}{\partial W_x} \right)$$

Backpropagation

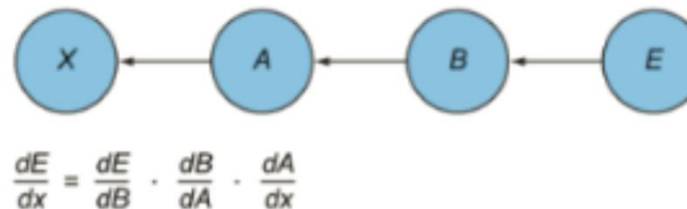
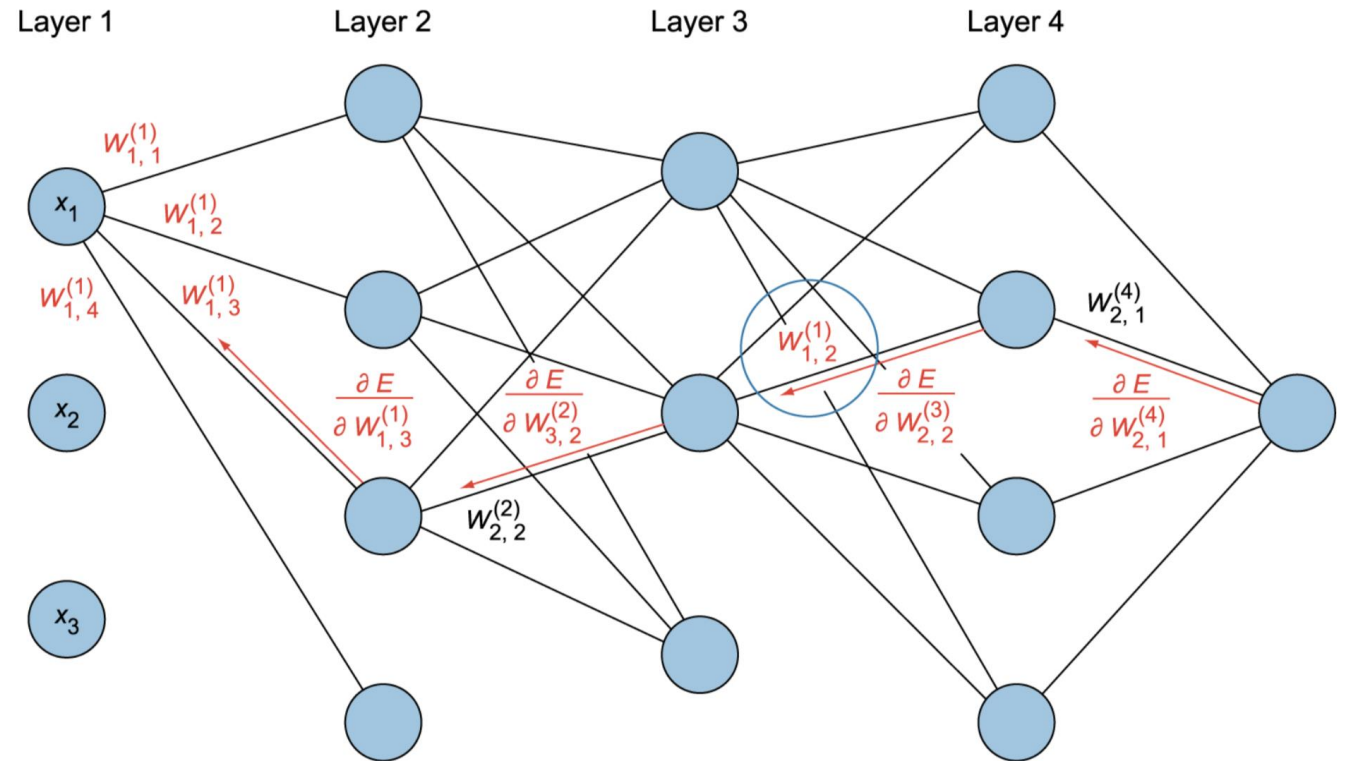
- Propagate derivative of the error w.r.t. to each specific weight from last layer to first layer to adjust weights
- Weights get updates using

$$(w_{next-step} = w_{current} + \Delta w)$$
- For multiple layers, we need to calculate the derivative of the error w.r.t to the weight. E.g.

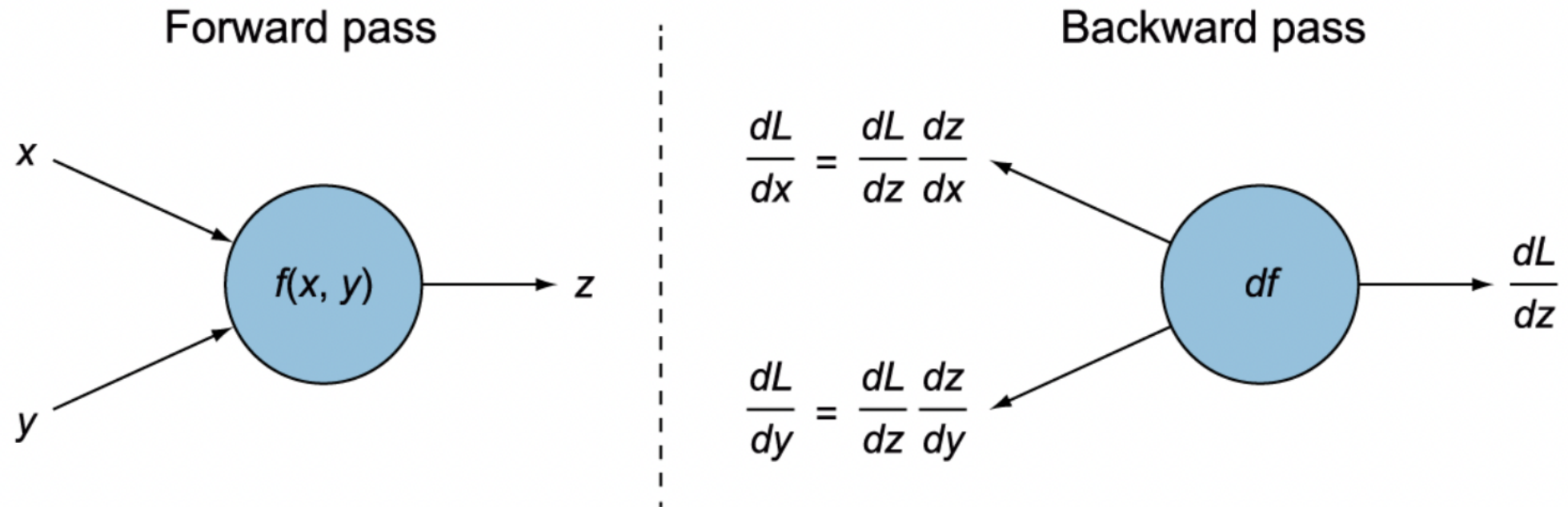
$$\frac{dE}{dw_{1,3}^{(1)}} = \frac{dE}{dw_{2,1}^{(4)}} * \frac{dw_{2,1}^{(4)}}{dw_{2,2}^{(3)}} * \frac{dw_{2,2}^{(3)}}{dw_{3,2}^{(2)}} * \frac{dw_{3,2}^{(2)}}{dw_{1,3}^{(1)}}$$

- Use of chain rule

$$\frac{d}{dx} f(g(x)) = f'(g(x))g'(x)$$



Backpropagation summary



Reference

- Book – Deep Learning for Vision Systems by Mohamad Elgendy, Manning Publication, November 2020